Marathon Finish Time Approximation

Predicting Race Performance Using FC-FFNN and RBF Networks

Saba Chichinadze

Kutaisi International University
Numerical Analysis, Computational Project 2

June 2026

Theorem Statement

A feedforward neural network with at least one hidden layer, a finite number of neurons, and a non-linear activation function can approximate any continuous function $f : K \subset \mathbb{R}^n \to \mathbb{R}$ on a compact domain K to any desired accuracy $\varepsilon > 0$:

$$\sup_{x \in K} \left| f(x) - F(x) \right| < \varepsilon$$

What the theorem does NOT tell us in practice:

- How many neurons are actually needed for a given problem
- Whether the training process will find the correct weights
- How long training will take or how much data is required
- Whether the network will generalise to unseen data
- Anything about robustness to noise or overfitting

2. FC-FFNN Implementation

Architecture: input(19) $\rightarrow$ 64 $\rightarrow$ 32 $\rightarrow$ output(1)

Key design choices:

- He initialisation: $w \sim \mathcal{N}\left(0, \sqrt{\frac{2}{n_{in}}}\right)$
- ReLU activation on hidden layers
- Linear output for regression
- Mini-batch SGD, batch size 256
- Learning rate 0.001, 100 epochs
- MSE loss: $\mathcal{L} = \frac{1}{m}\sum(y_i - t_i)^2$

Built entirely from scratch using **NumPy only**.
No TensorFlow or PyTorch used.

```
def forward(self, X):
    cur = X
    for i, (w, b) in enumerate(
            zip(self.W, self.b)):
        z = cur @ w + b
        a = relu(z) if i < len(
            self.W)-1 else z
        cur = a
    return cur

def backward(self, y):
    dA = (self.A[-1] - y) / m
    for i in reversed(range(
            len(self.W))):
        dZ = dA * relu_grad(self.Z[i]) \
             if i < len(self.W)-1 else dA
        dW = self.A[i].T @ dZ
        self.W[i] -= self.lr * dW
```

3. RBF Network Implementation

Architecture: 80 Gaussian centers, closed-form output weights

Key design choices:

- Centers via k-means clustering
- Gaussian activation:

$$\phi_j(x) = \exp\left(-\frac{\|x - c_j\|^2}{2\sigma^2}\right)$$

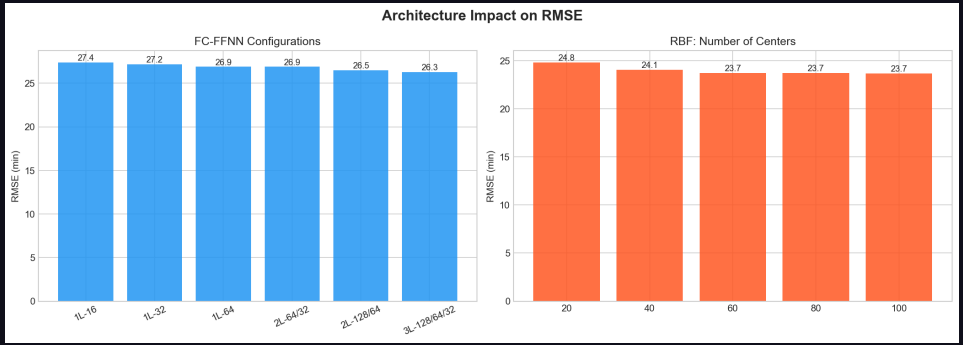
- Width σ = mean nearest-center distance
- Regularised least squares solution:

$$W = \left(\Phi^T\Phi + \lambda I\right)^{-1}\Phi^T y$$

- **No epochs, no learning rate needed**

```
def fit(self, X, y):
    self.centers = self._kmeans(
        X, self.n_centers)
    D = np.linalg.norm(
        X[:,None]-self.centers[None],
        axis=2)
    self.width = D.min(1).mean()
    Phi = np.exp(
        -D**2 / (2*self.width**2))
    Phi_b = np.hstack(
        [Phi, np.ones((len(Phi),1))])
    lam = 1e-3
    self.W = np.linalg.solve(
        Phi_b.T @ Phi_b +
        lam * np.eye(Phi_b.shape[1]),
        Phi_b.T @ y)
```

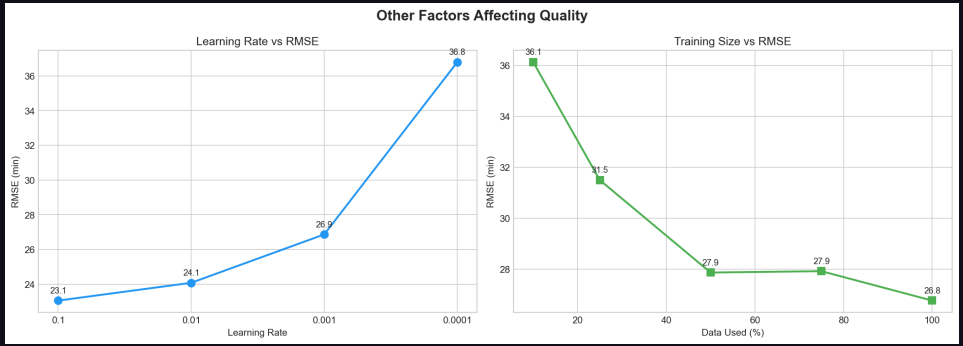
4. Network Architecture Impact



FC-FFNN findings: deeper and wider architectures reduce RMSE up to a point, after which gains diminish and overfitting risk increases.

RBF findings: more centers improve approximation quality but increase computation time. Beyond 80 centers improvement becomes marginal.

5. Other Factors Affecting Approximation Quality



Learning rate: 0.001 produced the best balance. Too high caused instability and divergence. Too low caused very slow convergence.

Training data size: performance improved consistently as more samples were used, confirming the importance of sufficient training data.

6. Application Problem

Problem: Marathon finish time prediction

Dataset: Run Club Marathon Performance Dataset

- 80,000 runners
- 19 numerical input features
- Target: finish time in minutes

Input features include:

- Age, BMI, resting heart rate
- Weekly mileage, long run distance
- Training adherence, streak length
- Recovery and motivation scores
- Injury count, previous marathons

Real world usage:

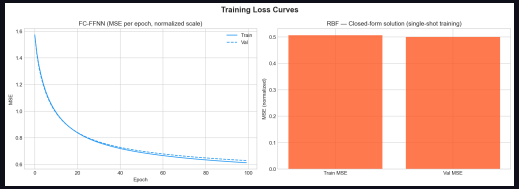
- Coaches set race expectations for athletes
- Runners design pacing strategy
- Training plans built from target finish time

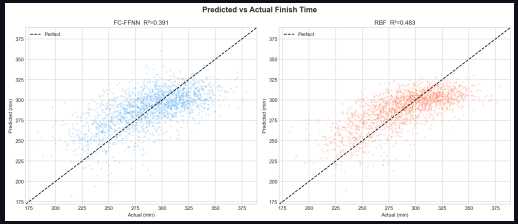
Distance extension via Riegel formula:

$$T_2 = T_1 \cdot \left(\frac{D_2}{D_1}\right)^{1.06}$$

Converts marathon prediction to 5km, 8km, 10km and half marathon estimates.

7. Training Results

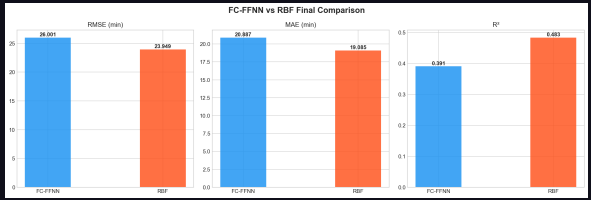




Left: FC-FFNN training and validation loss over 100 epochs. Loss decreases steadily confirming successful learning. RBF loss shown as a single value since it trains in one closed-form step.

Right: Predicted vs actual finish times on the validation set. Points close to the diagonal line indicate accurate predictions. Both models learn the general trend successfully.

8. FC-FFNN vs RBF Comparison





Model	RMSE	MAE	R^2
FC-FFNN	26.00 min	20.90 min	0.391
RBF	23.95 min	19.10 min	0.483

RBF outperformed FC-FFNN on all metrics despite training in a single step. FC-FFNN required 100 epochs of iterative backpropagation. Error distributions for both models are centred near zero, confirming unbiased predictions.

9. Summary

What was built

Two neural network architectures implemented from scratch in NumPy and applied to marathon finish time prediction from 80,000 runner records.

Key findings:

- RBF outperformed FC-FFNN (R^2: 0.483 vs 0.391)
- RBF achieves this in a single matrix operation
- FC-FFNN benefits from deeper architectures up to a point
- Learning rate 0.001 optimal for this problem
- More training data consistently improves both models

UAT confirmed in practice:

Both networks successfully approximated the nonlinear relationship between training characteristics and race performance, consistent with what the Universal Approximation Theorem guarantees in theory.

Interactive app:

Flask application running locally allows any runner to enter their characteristics and receive predictions across multiple distances.